

Task 7: Web Application Vulnerability Testing

Tools Used

- **Burp Suite Community Edition**
- **Target Application:** DVWA (Damn Vulnerable Web Application)
- **Alternative Tool:** OWASP ZAP (not used)

1. Understanding OWASP Top 10 Vulnerabilities

OWASP Top 10 is a list of the most dangerous and common security risks found in web applications. These vulnerabilities can allow attackers to steal data, break authentication, or take control of a website.

Some important OWASP vulnerabilities are:

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Authentication
- Security Misconfiguration
- Sensitive Data Exposure

In this lab, I mainly focused on:

- **SQL Injection**
- **Reflected XSS**

2. Setting up the Vulnerable Application (DVWA)

I installed and configured **DVWA** on my local system.

DVWA is a purposely vulnerable web application that helps in learning web security.

I logged in to DVWA and set:

Security Level → Low

This allows vulnerabilities to be exploited easily for learning purposes.

3. Intercepting Requests using Burp Suite

I opened **Burp Suite Community Edition** and started a temporary project.

Burp runs a proxy on:

127.0.0.1 : 8080

I opened the **Burp built-in browser** and accessed:

http://localhost/DVWA

Since Burp Proxy was running, all traffic between my browser and DVWA was intercepted and recorded in:

Burp → Proxy → HTTP History

This allowed me to see all requests like:

- Login requests
- SQL Injection page
- XSS page

4. Testing SQL Injection

I went to:

DVWA → SQL Injection

In the User ID input box, I entered the payload:

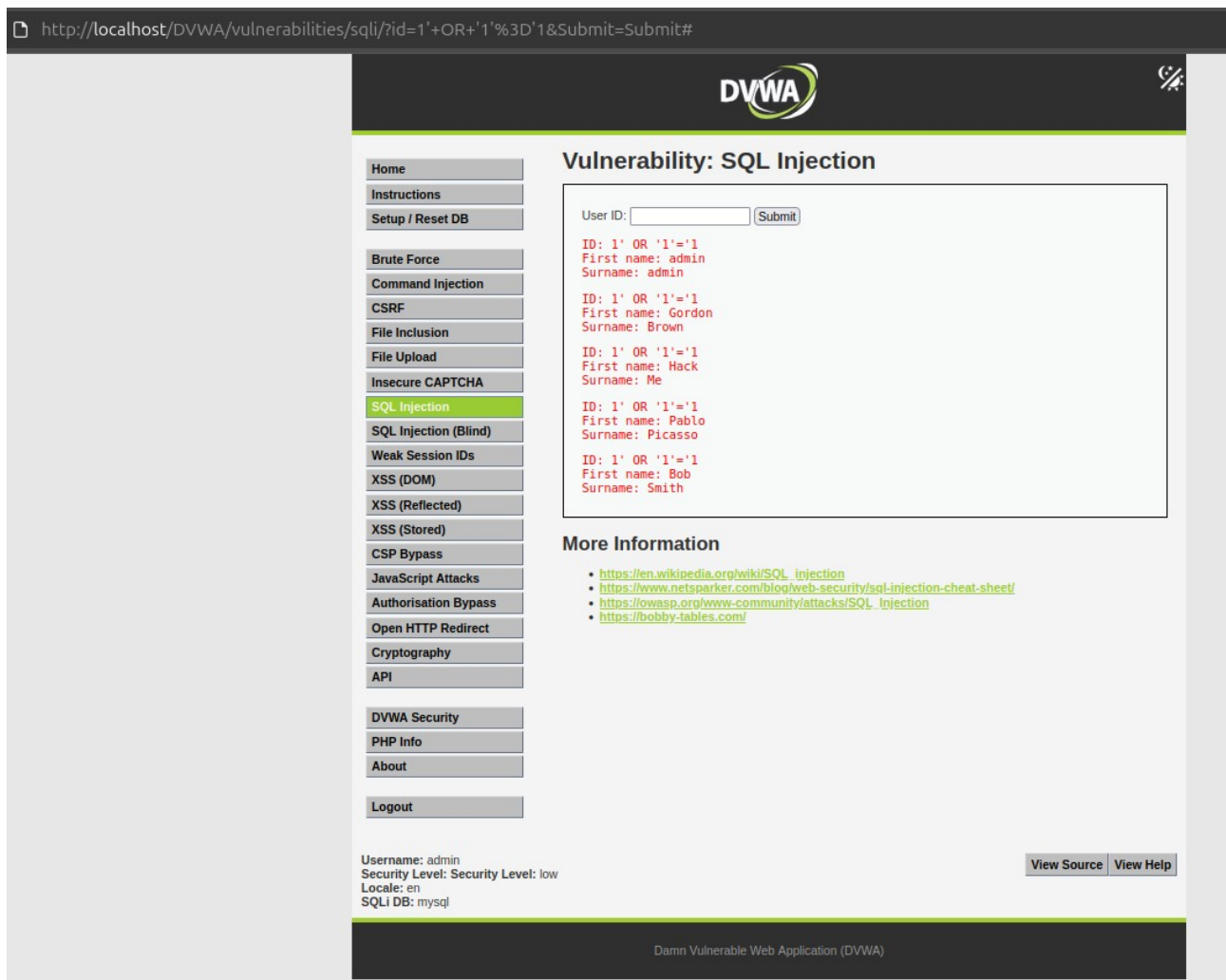
1' OR '1'='1

This caused the application to return multiple user records instead of one.

This shows that the SQL query was modified by user input, proving the application is vulnerable to SQL Injection.

Burp captured this request at:

/DVWA/vulnerabilities/sqli/



5. Testing Cross-Site Scripting (XSS)

I opened:

DVWA → XSS (Reflected)

I entered this payload:

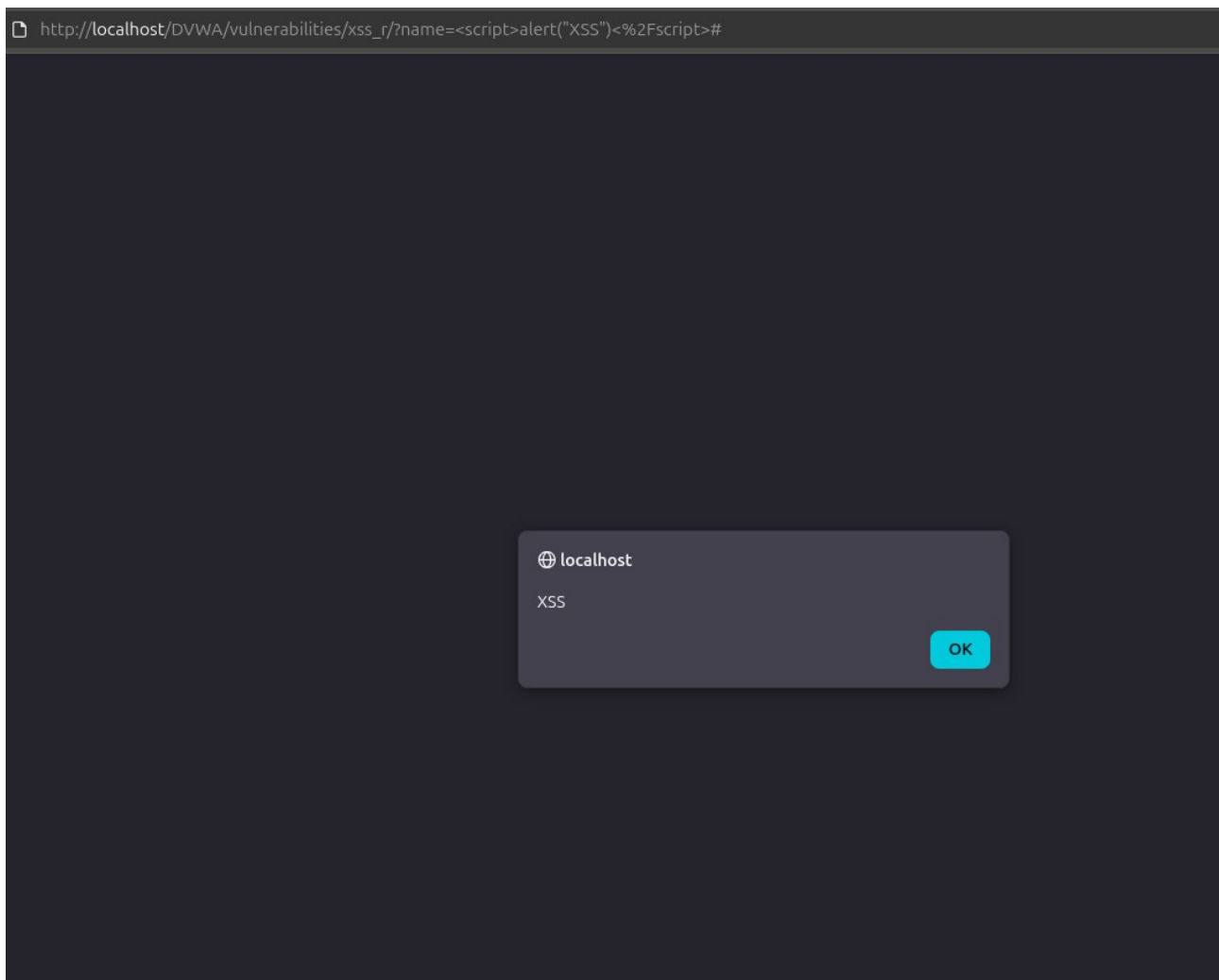
```
<script>alert("XSS")</script>
```

When I clicked submit, a popup appeared in the browser.

This means the input was executed as JavaScript, confirming a **Reflected XSS vulnerability**.

Burp recorded this request at:

```
/DVWA/vulnerabilities/xss_r/
```



6. Observing Application Responses

In **Burp** → **Proxy** → **HTTP History**, I observed:

- Requests to `/DVWA/vulnerabilities/sqli/`
- Requests to `/DVWA/vulnerabilities/xss_r/`
- Status code: 200 OK
- Host: 127.0.0.1
- Port: 8080

This confirmed that DVWA traffic was passing through Burp and vulnerabilities were successfully tested.

Burp Suite Community Edition v2025.12.5 - Temporary Project																
Dashboard		Target	Proxy	Intruder	Repeater	Collaborator	Sequencer	Decoder	Comparer	Logger	Organizer	Extensions	Learn			
Intercept		HTTP history		WebSockets history		Match and replace		Proxy settings								
Filter settings: Hiding CSS and image content; hiding specific extensions																
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listen
1	http://localhost	GET	/DVWA			301	564	HTML		301 Moved Permanently			127.0.0.1		22:58:24 27...	8080
2	http://localhost	GET	/DVWA/			302	665	HTML					127.0.0.1	security=impossi...	22:58:24 27...	8080
3	http://localhost	GET	/DVWA/login.php			200	1669	HTML	php	Login :: Damn Vulnera...			127.0.0.1		22:58:24 27...	8080
7	https://www.google.com	GET	/wampup.html			404	2268	HTML	html	Error 404 (Not Found)!!1			142.250.205.68		22:58:25 27...	8080
8	http://localhost	POST	/DVWA/login.php		✓	302	476	HTML	php			✓	127.0.0.1	PHPSESSID=104...	22:58:36 27...	8080
9	http://localhost	GET	/DVWA/index.php			200	6857	HTML	php	Welcome :: Damn Vul...			127.0.0.1		22:58:36 27...	8080
16	http://localhost	GET	/DVWA/security.php			200	5515	HTML	php	DVWA Security :: Da...			127.0.0.1		22:58:41 27...	8080
18	http://localhost	POST	/DVWA/security.php		✓	302	491	HTML	php				127.0.0.1	security=low; PH...	22:58:44 27...	8080
19	http://localhost	GET	/DVWA/security.php			200	5584	HTML	php	DVWA Security :: Da...			127.0.0.1		22:58:44 27...	8080
20	http://localhost	GET	/DVWA/vulnerabilities/sqli/			200	4919	HTML		Vulnerability: SQL Inje...			127.0.0.1		22:58:46 27...	8080
21	http://localhost	GET	/DVWA/vulnerabilities/sqli/?id=1%2...		✓	200	5267	HTML		Vulnerability: SQL Inje...			127.0.0.1		22:59:01 27...	8080
22	http://localhost	GET	/DVWA/vulnerabilities/xss_r/			200	5096	HTML		Vulnerability: Reflecte...			127.0.0.1		22:59:04 27...	8080
23	http://localhost	GET	/DVWA/vulnerabilities/xss_r/?name=...		✓	200	5143	HTML		Vulnerability: Reflecte...			127.0.0.1		22:59:27 27...	8080

7. Documented Vulnerabilities

Vulnerability	Page	Result
SQL Injection	/DVWA/vulnerabilities/sqli/	Database records were exposed
Reflected XSS	/DVWA/vulnerabilities/xss_r/	JavaScript executed in browser

8. Mitigation Suggestions

To protect real web applications from these attacks:

For SQL Injection

- Use prepared statements
- Validate and sanitize all user inputs
- Use parameterized queries

For XSS

- Encode output properly
- Validate input
- Use Content Security Policy (CSP)

General

- Keep software updated
- Apply secure coding practices

- Use a Web Application Firewall (WAF)

9. Conclusion

In this lab, I learned how attackers exploit web vulnerabilities and how Burp Suite can be used to detect them. By testing SQL Injection and XSS on DVWA, I understood how insecure input handling can lead to serious security issues. This experiment helped me understand real-world web security testing in a practical way.